Practical:08 **Date:08-10-2026**

Hyperparameter Optimization - I

Procedure:

Step 1: Import the required libraries for data analysis, model building, and hyperparameter optimization.

Step 2: Load the inbuilt datasets for regression and classification problems.

Step 3: Separate the independent variables (X) and dependent variable (y).

Step 4: Split the dataset into training and testing sets using train_test_split().

Step 5: Select Ridge Regression for the regression problem and Logistic Regression for the classification problem.

Step 6: Define the hyperparameter values or distributions to be tested for Grid Search and Random Search.

Step 7: Apply GridSearchCV() and RandomizedSearchCV() with suitable cross-validation and scoring metrics.

Step 8: Fit the search methods using the training data.

Step 9: Identify the best hyperparameters and cross-validation scores using best_params_ and best_score_.

Step 10: Evaluate the models using Mean Absolute Error (MAE) for regression and accuracy for classification.

Step 11: Display and compare the results obtained using Grid Search and Random Search.

Solution:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
# Load the inbuilt dataset
data = load_diabetes()
X, y = data.data, data.target
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Define the model
model = Ridge()
# Define hyperparameter values
param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100]
}
# Perform Grid Search
grid = GridSearchCV(
    model, param_grid, cv=5,
    scoring='neg_mean_absolute_error'
)
```

```
grid.fit(X_train, y_train)
# Best results
print("Best Parameters:", grid.best_params_)
print("Best CV MAE:", -grid.best_score_)
# Test the best model
y_pred = grid.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred))
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error
from scipy.stats import loguniform
# Load the dataset
data = load_diabetes()
X, y = data.data, data.target
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Define the model
model = Ridge()
# Define the search space
param_dist = {
    'alpha': loguniform(0.001, 100)
}
# Perform Random Search
random_search = RandomizedSearchCV(
    model, param_dist, n_iter=20, cv=5,
    scoring='neg_mean_absolute_error',
    random_state=42
)
random_search.fit(X_train, y_train)
# Best results
print("Best Parameters:", random_search.best_params_)
print("Best CV MAE:", -random_search.best_score_)
```

```
# Test the best model
y_pred = random_search.predict(X_test)
print("Test MAE:", mean_absolute_error(y_test, y_pred))
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Load the dataset
data = load_breast_cancer()
X, y = data.data, data.target
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Create a pipeline
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000)
)
```

```
# Define hyperparameter values
param_grid = {
    'logisticregression__C': [0.01, 0.1, 1, 10, 100]
}
# Perform Grid Search
grid = GridSearchCV(
    model, param_grid, cv=5, scoring='accuracy'
)
grid.fit(X_train, y_train)
# Best results
print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
# Test the best model
y_pred = grid.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred))
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from scipy.stats import loguniform
```

```
# Load the dataset
data = load_breast_cancer()
X, y = data.data, data.target
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Create a pipeline
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000)
)
# Define the search space
param_dist = {
    'logisticregression__C': loguniform(0.001, 100)
}
# Perform Random Search
random_search = RandomizedSearchCV(
    model, param_dist, n_iter=20, cv=5,
    scoring='accuracy', random_state=42
)
```

```
random_search.fit(X_train, y_train)
# Best results
print("Best Parameters:", random_search.best_params_)
print("Best CV Accuracy:", random_search.best_score_)
# Test the best model
y_pred = random_search.predict(X_test)
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
Best Parameters: {'alpha': 0.01}
Best CV MAE: 45.42747579646327
Test MAE: 42.836887676091756
Best Parameters: {'alpha': np.float64(0.033205591037519584)}
Best CV MAE: 45.37958400935594
Test MAE: 42.866432270244125
Best Parameters: {'logisticregression__C': 0.1}
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9736842105263158
Best Parameters: {'logisticregression__C': np.float64(0.9846738873614566)}
```

```
Best CV Accuracy: 0.9802197802197803
Test Accuracy: 0.9824561403508771
```